

ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І. І. МЕЧНИКОВА  
ФАКУЛЬТЕТ МАТЕМАТИКИ, ФІЗИКИ І ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ  
КАФЕДРА ОПТИМАЛЬНОГО КЕРУВАННЯ І ЕКОНОМІЧНОЇ КІБЕРНЕТИКИ

## **Паралельні алгоритми обчислювальної математики**

### **Звіт**

**з лабораторної роботи №2**

**Паралельне сортування парно-непарними перестановками з OpenMP**

Виконав: студент 1 курсу магістратури  
спеціальності 113 Прикладна математика  
Чачко Натан Леонідович

2025 р.

Послідовний алгоритм сортування парно-непарними перестановками може бути реалізований таким чином:

```

1 for (phase = 0; phase < n; phase++)
2     if (phase % 2 == 0)
3         for (i = 1; i < n; i += 2)
4             if (a[i-1] > a[i]) Swap(&a[i-1], &a[i]);
5     else
6         for (i = 1; i < n-1; i += 2)
7             if (a[i] > a[i+1]) Swap(&a[i], &a[i+1]);

```

Разглянемо короткий приклад. Нехай,  $a = (9, 7, 8, 6)$ . Фази алгоритма наведені в Таблиці 2.

| Phase |                                               |
|-------|-----------------------------------------------|
| 0     | 9 ↔ 7      8 ↔ 6<br>7      9      6      8    |
| 1     | 7      9 ↔ 6      8<br>7      6      9      8 |
| 2     | 7 ↔ 6      9 ↔ 8<br>6      7      8      9    |
| 3     | 6      7 ↔ 8      9<br>6      7      8      9 |

Табл. 2: Сортування парно-непарними перестановками

Сортування з парно-непарними перестановками можна розпаралелити, наприклад, так:

```

1 for (phase = 0; phase < n; phase++) {
2     if (phase % 2 == 0)
3         #pragma omp parallel for num_threads(thread count) \
4             default(none) shared(a, n) private(i, tmp)
5             for (i = 1; i < n; i += 2) {
6                 if (a[i-1] > a[i]) {
7                     tmp = a[i-1]; a[i-1] = a[i]; a[i] = tmp;
8                 }
9             }
10    else
11        #pragma omp parallel for num_threads(thread count) \
12            default(none) shared(a, n) private(i, tmp)
13            for (i = 1; i < n-1; i += 2) {
14                if (a[i] > a[i+1]) {
15                    tmp = a[i+1]; a[i+1] = a[i]; a[i] = tmp;
16                }
17            }
18 }

```

Дві проблеми:

- синхронізація фаз;
- накладні витрати: створення та знищення паралельних потоків для кожної ітерації зовнішнього циклу.

Інший варіант паралельного алгоритму сортування:

```
1 #pragma omp parallel num_threads(thread_count) \  
2   default(none) shared(a, n) private(i, tmp, phase)  
3 for (phase = 0; phase < n; phase++) {  
4   if (phase % 2 == 0)  
5     #pragma omp for  
6     for (i = 1; i < n; i += 2) {  
7       if (a[i-1] > a[i]) {  
8         tmp = a[i-1];  
9         a[i-1] = a[i];  
10        a[i] = tmp;  
11      }  
12    }  
13  else  
14    #pragma omp for  
15    for (i = 1; i < n-1; i += 2) {  
16      if (a[i] > a[i+1]) {  
17        tmp = a[i+1];  
18        a[i+1] = a[i];  
19        a[i] = tmp;  
20      }  
21    }  
22 }
```

Тут директива for, на відміну від директиви parallel for, не створює потоки. Вона використовує ті потоки, які вже були створені директивою parallel.

## Завдання

Провести обчислювальний експеримент та заповнити таблицю 3.

Побудувати графіки залежності часу сортування від кількості потоків для різних  $n$ .

До звіту включити відомості про обчислювальну систему та компілятор, що використовувались для обчислень.

```
1 #include <math.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <omp.h>
5
6 int main() {
7     long long sizes[] = {4, 5, 6, 7};
8     int thread_counts[] = {1, 2, 4, 8};
9
10    for (int s = 0; s < 4; s++) {
11        long long n = pow(10, sizes[s]);
12        int *a = malloc(sizeof(int) * n);
13
14        // Заповнення масиву випадковими значеннями
15        for (long long i = 0; i < n; i++) {
16            a[i] = rand();
17        }
18
19        printf("\n=== n = %lld ===\n", n);
20
21        for (int t = 0; t < 4; t++) {
22            int thread_count = thread_counts[t];
23
24            printf("Sorting started: threads = %d | progress [", thread_count);
25            fflush(stdout); // Примусовий вивід повідомлення на екран
26            // Копія початкового масиву для кожного запуску
27            int *a_copy = malloc(sizeof(int) * n);
28            for (long long i = 0; i < n; i++) a_copy[i] = a[i];
29
30            double s_time = omp_get_wtime();
31
32            // Кожна фаза сортування виконується у паралельному блоці
33            #pragma omp parallel num_threads(thread_count) default(none) shared(a_copy, n)
34            {
35                int i, tmp;
36                for (int phase = 0; phase < n; phase++) {
37                    // Вивід прогресу у відсотках (тільки з одного потоку)
38                    #pragma omp master
39                    if (phase % (n / 10) == 0) {
40                        int percent = (int)((100.0 * phase) / n);
41                        printf("%d%% ", percent);
42                        fflush(stdout); // Примусовий вивід відсотків на екран
43                    }
44
45                    if (phase % 2 == 0) {
46                        #pragma omp for
47                        for (i = 1; i < n; i += 2) {
48                            if (a_copy[i - 1] > a_copy[i]) {
49                                tmp = a_copy[i - 1];
50                                a_copy[i - 1] = a_copy[i];
51                                a_copy[i] = tmp;
52                            }
53                        }
54                    } else {
```

```

55 #pragma omp for
56         for (i = 1; i < n - 1; i += 2) {
57             if (a_copy[i] > a_copy[i + 1]) {
58                 tmp = a_copy[i + 1];
59                 a_copy[i + 1] = a_copy[i];
60                 a_copy[i] = tmp;
61             }
62         }
63     }
64 }
65
66
67     double f_time = omp_get_wtime();
68     printf("100%% | time = %.6f sec\n", f_time - s_time);
69     free(a_copy);
70 }
71     free(a);
72 }
73     return 0;
74 }

```

Зі збільшенням значень  $n$  час роботи програми зростає дуже стрімко. Тому було прийнято рішення використовувати відносно малі значення:  $10^4$ ,  $10^5$ ,  $10^6$ . Для  $n = 10^7$  програма за 8 годин виконала менше ніж 50% роботи.

Також результат змінювався для різних запусків програми. Нижче наведено вивід програми для трьох різних запусків.

### Вивід програми (1):

```

=== n = 10000 ===
Sorting started: threads = 1 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 0.040000 sec
Sorting started: threads = 2 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 0.151000 sec
Sorting started: threads = 4 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 0.298000 sec
Sorting started: threads = 8 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 0.525000 sec

=== n = 100000 ===
Sorting started: threads = 1 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 6.052000 sec
Sorting started: threads = 2 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 3.982000 sec
Sorting started: threads = 4 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 3.394000 sec
Sorting started: threads = 8 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 4.685000 sec

=== n = 1000000 ===
Sorting started: threads = 1 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 1040.593000 sec
Sorting started: threads = 2 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 657.147000 sec
Sorting started: threads = 4 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 521.499000 sec
Sorting started: threads = 8 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 319.403000 sec

```

### Вивід програми (2):

```

=== n = 10000 ===
Sorting started: threads = 1 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 0.055000 sec
Sorting started: threads = 2 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 0.202000 sec
Sorting started: threads = 4 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 0.407000 sec
Sorting started: threads = 8 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 0.754000 sec

=== n = 100000 ===
Sorting started: threads = 1 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 6.330000 sec
Sorting started: threads = 2 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 6.601000 sec
Sorting started: threads = 4 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 5.017000 sec
Sorting started: threads = 8 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 6.469000 sec

```

```

=== n = 1000000 ===
Sorting started: threads = 1 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 679.356000 sec
Sorting started: threads = 2 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 516.924000 sec
Sorting started: threads = 4 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 396.899000 sec
Sorting started: threads = 8 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 326.774000 sec

```

### Вивід програми (3):

```

=== n = 10000 ===
Sorting started: threads = 1 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 0.048000 sec
Sorting started: threads = 2 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 0.141000 sec
Sorting started: threads = 4 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 0.268000 sec
Sorting started: threads = 8 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 0.549000 sec

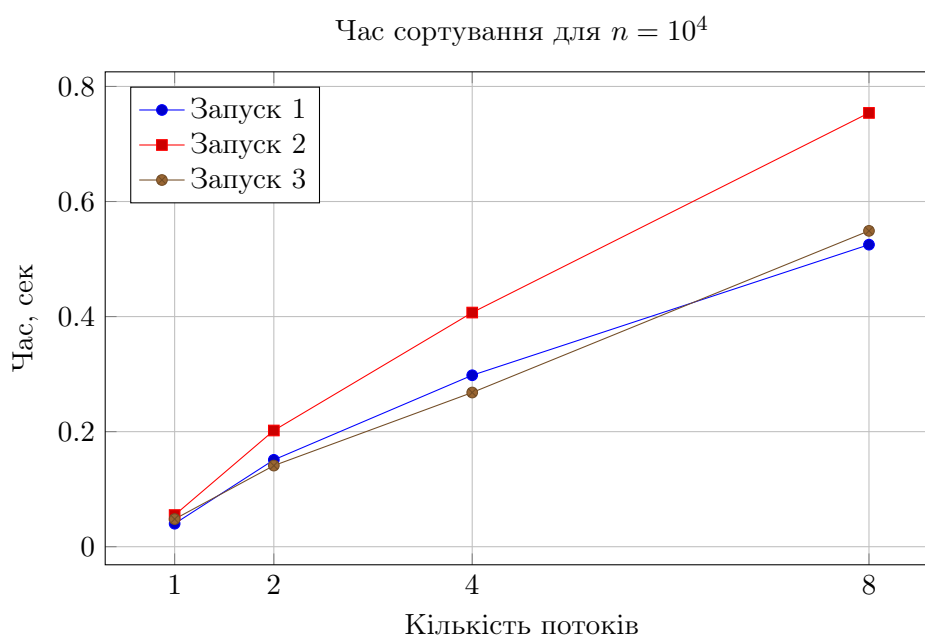
=== n = 100000 ===
Sorting started: threads = 1 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 6.085000 sec
Sorting started: threads = 2 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 4.012000 sec
Sorting started: threads = 4 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 3.392000 sec
Sorting started: threads = 8 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 4.762000 sec

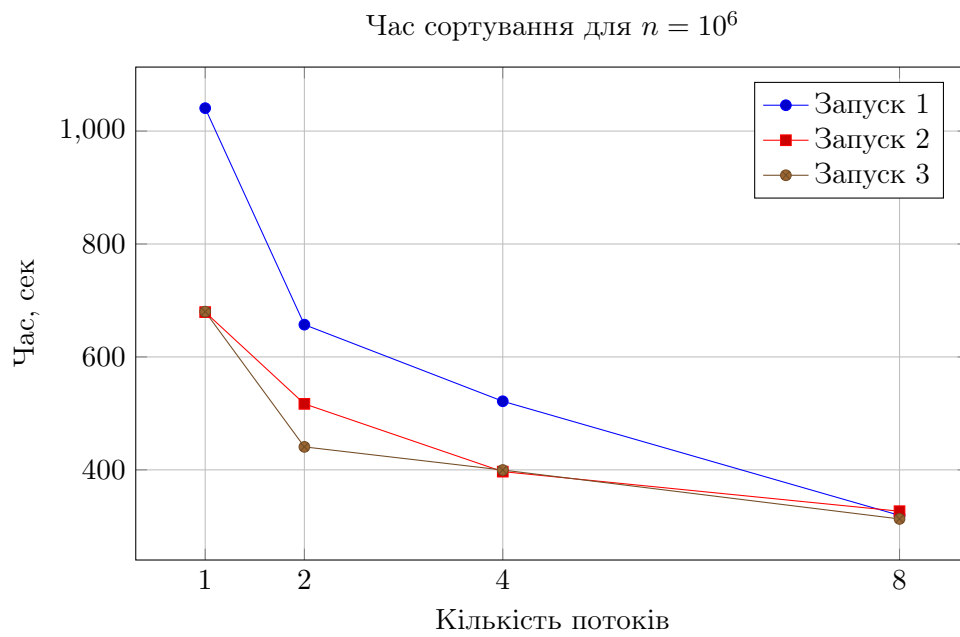
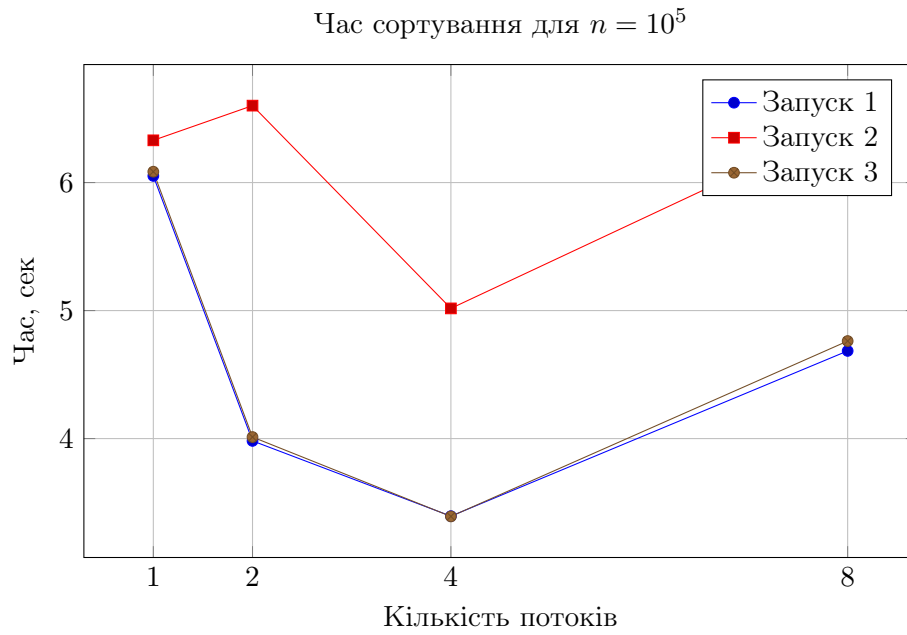
=== n = 1000000 ===
Sorting started: threads = 1 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 680.195000 sec
Sorting started: threads = 2 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 440.720000 sec
Sorting started: threads = 4 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 399.732000 sec
Sorting started: threads = 8 | progress [0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100%] | time = 313.105000 sec

```

| $n$    | 1 потік               | 2 потоки              | 4 потоки              | 8 потоків             |
|--------|-----------------------|-----------------------|-----------------------|-----------------------|
| $10^4$ | 0.040 / 0.055 / 0.048 | 0.151 / 0.202 / 0.141 | 0.298 / 0.407 / 0.268 | 0.525 / 0.754 / 0.549 |
| $10^5$ | 6.052 / 6.330 / 6.085 | 3.982 / 6.601 / 4.012 | 3.394 / 5.017 / 3.392 | 4.685 / 6.469 / 4.762 |
| $10^6$ | 1040 / 679 / 680      | 657 / 516 / 440       | 521 / 396 / 399       | 319 / 326 / 313       |

Табл. 3: Час сортування масиву цілих чисел розміру  $n$  паралельною програмою з різною кількістю потоків (три запуски через «/»).





Обчислення проводилися з використанням компілятора GCC (MinGW) на комп'ютері з операційною системою Windows 10 та наступними характеристиками:

CPU:

11th Gen Intel(R) Core(TM) i7-1185G7 @ 3.00GHz 1.80 GHz

Base speed: 1,80 GHz

Cores: 4

Logical processors: 8